Rangmanch

Empowering Artisans: A Local Artisan Marketplace

Team: AC-DC
Rachit Tiwari (24BAI10309)
Shaikh Mohammad Warsi (24BAI10046)

Hack Matrix Hackathon
VIT Bhopal

April 2, 2026

Contents

1 Executive Summary **3**

2 Problem Statement & Market Gaps **3**

3 Technical Ecosystem **3**
3.1 Frontend Architecture 3
3.2 Backend & Database Ecosystem 4

4 Data Architecture (PostgreSQL) **4**
4.1 Core Tables & Relationships 5

5 Security & Authentication Workflow **5**
5.1 Authentication Flow 5
5.2 Access Control (Row-Level Security) 6

6 Key Functional Modules **6**
6.1 The Heritage Navigator (Map Component) 6
6.2 The Direct-to-Consumer (D2C) Marketplace 7
6.3 Artisan Trust Verification 7

7 Performance & Scalability Design **7**

8 Development Roadmap **7**

9 Coding Conventions **8**

10 Conclusion **8**

1 Executive Summary

Rangmanch is a specialized digital marketplace designed to bridge the gap between rural artisans and global consumers, operating as a next-generation **"Legacy-as-a-Service"** platform. Its primary goal is to provide Indian artisans with a high-performance, cinematic marketplace using modern cloud infrastructure. By providing a decentralized, creator-first ecosystem, the platform brings previously localized art forms to a national and international audience.

By migrating from standard NoSQL concepts to a **Relational PostgreSQL** system and leveraging **Supabase**, the platform ensures ACID compliance for transactions and enterprise-grade security for user data. This fundamental shift in architecture guarantees high data integrity, robust querying capabilities, and a scalable foundation that can support millions of concurrent connections during peak e-commerce seasons. Rangmanch empowers low-income artisans to list their handmade goods seamlessly, connecting them directly with conscious buyers and supporting NGOs, all while making traditional crafts financially viable for the modern digital economy.

2 Problem Statement & Market Gaps

Rural artisans are the custodians of high-quality, authentic handmade goods, representing centuries of cultural heritage. However, without an accessible digital sales channel, they are entirely dependent on local foot traffic or exploitative middlemen, who often siphon off up to 70% of the actual profit margins.

- **High Barriers to Entry:** Current platforms (Etsy, Amazon Karigar) require high technical literacy, GST registration upfront, and complex inventory management systems that alienate grassroots creators.
- **Financial Exploitation:** High commission fees, hidden listing charges, and delayed payout structures eat significantly into the artisan's already narrow profit margins, discouraging them from pursuing their craft full-time.
- **Communication Deficit:** There is a distinct lack of direct, native communication channels for custom orders, bulk negotiations, or personalized sizing.
- **Loss of Cultural Context:** Standard e-commerce platforms treat handmade crafts as commodity goods, stripping away the rich legacy and storytelling that conscious consumers value.

Rangmanch solves this by providing a **"Zero-Barrier"** entry point with simplified mobile-first listings, localized regional discovery filters, and a dual-action commerce model combining secure checkout with direct WhatsApp communication.

3 Technical Ecosystem

Rangmanch is built on a modern, decoupled architecture ensuring robust scalability, lightning-fast load times even on low-bandwidth rural networks, and secure financial transactions.

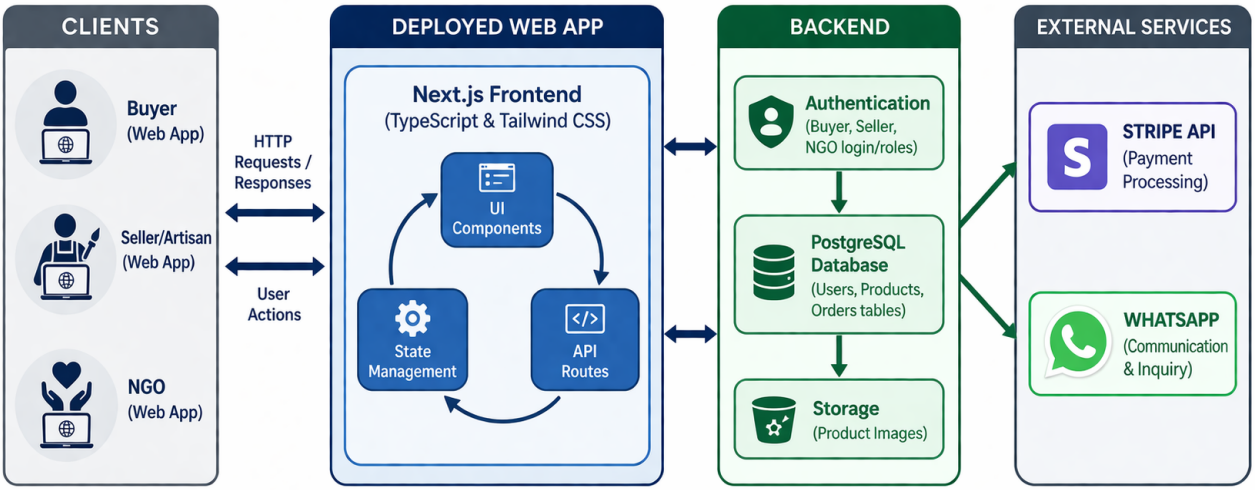
3.1 Frontend Architecture

- **Framework:** Next.js 14/15 (leveraging Server Components for aggressive SEO optimization and instantaneous initial page loads via Edge caching).

- **UI Library:** React 19 combined with Tailwind CSS allows for highly responsive, utility-first styling that adapts flawlessly to mobile, tablet, and desktop viewports.
- **Animations & Micro-interactions:** We utilize GSAP, Framer Motion, and React Spring (Parallax) to deliver a cinematic, high-fidelity user experience that highlights the intricate details of the crafts.
- **Client-Side Logic:** Lucide React handles lightweight SVG iconography, while React Leaflet powers the interactive, data-driven cultural map of India.

3.2 Backend & Database Ecosystem

- **Server Infrastructure:** A Node.js and Express.js (TypeScript) environment running a custom middleware stack designed for high-performance, asynchronous request handling.
- **Authentication:** Supabase Auth (GoTrue) seamlessly handles user sessions, secure JWT issuance, social logins, and Role-Based Access Control (RBAC).
- **Database:** PostgreSQL (hosted on Supabase) ensures structured, relational data mapping with complete ACID compliance to prevent race conditions during high-demand sales.
- **ORM/Querying:** `pg` (Node-Postgres) facilitates direct, high-performance database interactions, bypassing the overhead of heavier ORMs.
- **Payments:** Dual gateway integration featuring Razorpay for localized Indian transactions (UPI, RuPay) alongside Stripe for global credit card processing.



4 Data Architecture (PostgreSQL)

Unlike traditional document-based storage, our schema utilizes **Relational Normalization** to ensure strict data integrity, eliminate data redundancy, and maintain complex relationships between artisans, their crafts, and regional data.

4.1 Core Tables & Relationships

`users` (Profile Table)

Linked via `supabase.auth.uuid` for secure identity mapping.

- **Columns:** `full_name`, `avatar_url`, `role` (ENUM: `artisan`, `buyer`, `ngo`), `location` (Geo-coordinates), `craft_specialization`.
- **Details:** Acts as the central hub for all user activities, securely storing both contact information and platform-specific roles.

`products` Table

- **Foreign Key:** `artisan_id` strictly references `users(id)` with cascading deletes.
- **Columns:** `title`, `slug`, `description`, `base_price`, `stock_quantity`, `material_used`.
- **Details:** Contains the core inventory data. The `slug` ensures SEO-friendly URLs, while `stock_quantity` is dynamically updated via database triggers to prevent overselling.

`states_cultural_data` Table

Provides crucial metadata for the interactive Heritage Navigator map.

- **Columns:** `state_name`, `legacy_description`, `image_banner`, `craft_list` (JSONB).
- **Details:** The `JSONB` column allows for flexible, easily updatable lists of indigenous crafts specific to a geographical region without requiring frequent schema migrations.

`orders` Table

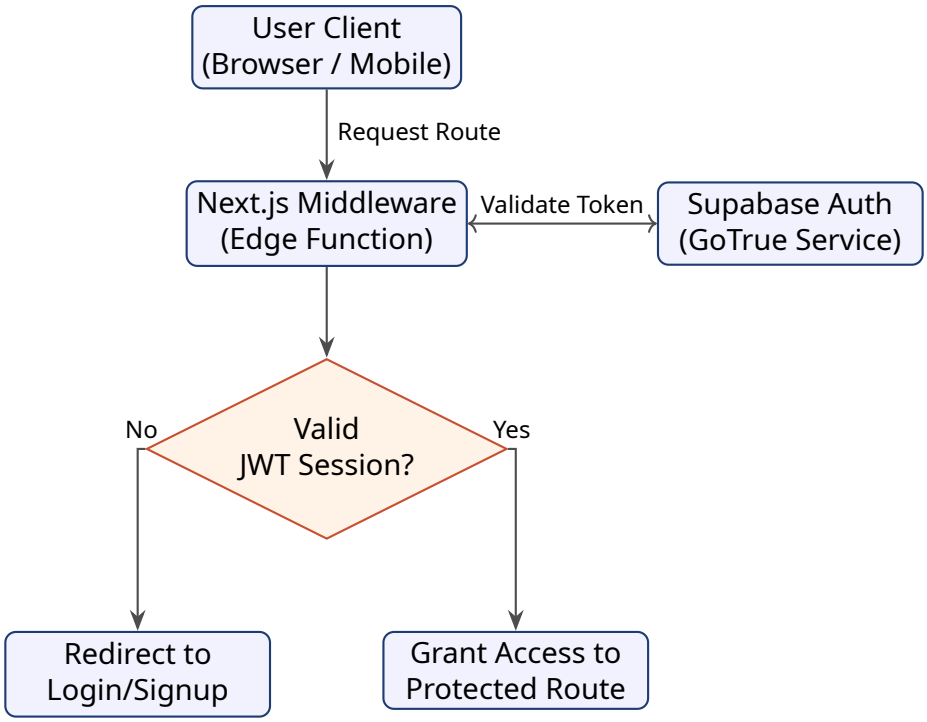
- **Foreign Keys:** `buyer_id`, `artisan_id`, `product_id`.
- **Columns:** `status` (ENUM: `pending`, `shipped`, `delivered`, `cancelled`), `transaction_id`, `payout_status`.
- **Details:** Central ledger for all commerce, meticulously tracking the lifecycle of a purchase from initial checkout to final artisan payout.

5 Security & Authentication Workflow

Rangmanch rigorously implements the **Supabase Auth + Row-Level Security (RLS)** paradigm. This approach effectively shifts access control to the database layer, protecting user data from API-level vulnerabilities and restricting unauthorized database operations natively.

5.1 Authentication Flow

- **Login:** Users authenticate securely via `supabase.auth.signInWithPassword` or seamless OTP-based mobile login.
- **Token Management:** Upon successful authentication, Supabase issues short-lived JWTs alongside secure refresh tokens.
- **Session Persistence:** Next.js edge middleware intercepts requests to evaluate the Supabase session cookie securely, ensuring that sensitive routes (like the Artisan Dashboard or NGO Verification Panel) remain strictly protected from unauthenticated access.



5.2 Access Control (Row-Level Security)

To prevent data contamination and ensure privacy, Postgres RLS policies are applied directly to the tables:

- **Buyers:** Granted read-only access to public `products` tables and `INSERT/UPDATE` access exclusively to their own rows in the `orders` table.
- **Artisans:** Granted `INSERT/UPDATE/DELETE` privileges strictly for their own `product` listings. They are cryptographically blocked from reading or modifying listings owned by competitors.
- **NGOs:** Granted specialized `VIEW` access to analyze aggregated regional craft statistics, evaluate artisan applications, and issue verification badges.

6 Key Functional Modules

6.1 The Heritage Navigator (Map Component)

The visual centerpiece of the Rangmanch platform is the interactive Heritage Navigator.

- **Logic:** Utilizes `react-leaflet` to render a dynamic, topologically accurate GeoJSON map of India.
- **Interaction:** Clicking a specific state triggers an optimized database query to the `states_cultural_da` table, instantly fetching and returning a curated list of local crafts (e.g., Terracotta in West Bengal, Blue Pottery in Rajasthan) and active artisans in that region.
- **Aesthetic:** Implemented with multi-layered Parallax effects using React Spring to make the geographic exploration feel tactile, educational, and highly immersive.

6.2 The Direct-to-Consumer (D2C) Marketplace

A frictionless shopping experience optimized for both standard web browsing and mobile interactions.

- **Product Discovery:** Features "Smart Search" utilizing PostgreSQL's native **Full-Text Search (`tsvector`).** This allows the system to intelligently interpret search queries, finding products by material, state, or technique even if the user misspells certain terms.
- **Inventory Sync:** Real-time stock updates are handled via backend database transactions to guarantee accuracy and prevent the overselling of unique, one-of-a-kind handmade items.

6.3 Artisan Trust Verification

To combat the proliferation of counterfeit machine-made goods masquerading as handmade items, Rangmanch features a specialized NGO portal. This portal allows registered NGOs to review, physically audit, and approve artisan certifications. Successfully verified artisans receive a prominent "Blue Check" badge on their profiles and listings, visually signaling absolute authenticity to prospective buyers.

7 Performance & Scalability Design

To effortlessly handle sudden spikes in traffic during festive seasons (like Diwali or Navratri), Rangmanch employs several aggressive optimization layers:

- **Database Indexing:** B-Tree indexing is applied on primary/foreign keys (e.g., `product_id`, `artisan_id`), while GIN (Generalized Inverted Index) is utilized on the `craft_list` JSONB column and text fields. This guarantees sub-millisecond query results even with millions of rows.
- **Serverless Readiness:** The Express backend is designed to be completely stateless, securely handling JWTs instead of server-side sessions, allowing it to scale horizontally across multiple instances instantly via load balancers.
- **Edge Caching & CDNs:** Utilizes Next.js **ISR** (Incremental Static Regeneration) to cache heavy product pages globally at the edge. The system updates these cached pages seamlessly in the background every few minutes, guaranteeing ultra-fast load times without sacrificing data freshness.

8 Development Roadmap

The project's evolution is structured into four distinct, strategically prioritized phases:

- **Phase 1 (Stabilization):** Complete, rigorous migration of all legacy MongoDB/Firebase data remnants to the robust PostgreSQL schema. Establishing base RLS policies.
- **Phase 2 (Visuals & UI/UX):** Enhancement of the Parallax scrolling experience (`@react-spring/paralla` to amplify the artisan storytelling sections, alongside the implementation of the Heritage Map.
- **Phase 3 (Mobile & Accessibility):** Progressive Web App (PWA) implementation specifically tailored for artisans in remote rural areas suffering from low-bandwidth connectivity, allowing for offline product draft creation.

- **Phase 4 (AI Integration):** Implementation of automated craft recognition, quality assurance, and image auto-tagging utilizing Supabase Edge Functions linked with lightweight machine learning models.

9 Coding Conventions

To maintain enterprise-grade codebase quality, scalability, and developer sanity, the team strictly adheres to the following conventions:

- **TypeScript:** Strict mode enabled globally. Explicit interfaces and types must be declared and utilized for all API responses, database schemas, and React component props.
- **Commit Logic:** Enforced Atomic commits adhering to conventional commit guidelines (`feat:`, `fix:`, `chore:`, `docs:`) for clear, traversable, and automated version history.
- **Environment Safety:** All Supabase keys, database URLs, and payment gateway secrets must remain securely in `.env.local` files. These are strictly ignored by Git and managed securely via the hosting platform's environment variables dashboard.

10 Conclusion

"By utilizing Supabase for identity management and PostgreSQL for relational depth, Rangmanch creates a production-ready infrastructure that matches the complexity and richness of Indian craftsmanship."

Rangmanch acts as a vital digital bridge for rural artisans, transforming every local smartphone into a global storefront. By enabling direct commerce through secure localized payments and familiar communication tools like WhatsApp, the platform cuts through the noise of modern mass manufacturing. It empowers craftsmen to preserve their heritage, share their stories, and achieve true modern financial independence, ensuring that ancient traditions thrive powerfully in the digital age.